# GOVERNMENT POLYTECHNIC, PUNE

(An Autonomous Institute of Government of Maharashtra)

**Programme:** Electronics & Telecommunication

**Academic Year:** 2025-26

**Course Name:** Microcontroller & applications

**Div:** L2

**Semester:** EVEN 2025

## A Microproject on:

## ISP Programmer of 8051 using Arduino Nano

---

## Submitted by:

| Sr no. | Enrolment no. | Name of the student |
|---|---|---|
| 1. | 2403069 | Vedant G Kulkarni |
| 2. | 2403081 | Pranil S. Nikam |
| 3. | 2403101 | Aditya S. Sapatnekar |
| 4. | 2403119 | Adinath B. Shete |

**Under the guidance of:**

Mrs. P.M. Zilpe (Course teacher)

## Introduction:

The 8051 microcontroller is widely used in embedded systems due to its simple architecture, low cost, and easy programming. During development, the microcontroller needs to be programmed many times. Earlier, the chip had to be removed from the circuit for programming, which was time-consuming.

To solve this problem, In-System Programming (ISP) is used. ISP allows the microcontroller to be programmed directly on the development board without removing it from the circuit. In this microproject, an 8051 ISP programmer is used to upload the HEX file from a computer to the microcontroller. This makes programming faster, easier, and more efficient for developing embedded system applications.

## Course outcomes addressed:

- Understand the architecture of the 8051 microcontroller.
- Learn the concept and importance of In-System Programming (ISP).
- Develop the ability to upload a HEX file into the microcontroller using an ISP programmer.

## Components required:

| Sr. No. | Component | Specification |
|---|---|---|
| 1. | Breadboard | 17 x 5.5 cm |
| 2. | Arduino Nano | Dev board, 16 MHz External Clock |
| 3. | Atmel AT89S52 | DIP Package |
| 4. | Connecting Wires | Single strand Teflon coating (0.6mm diameter) |
| 5. | Capacitors | 2 x 22pF |
| 6. | Resistor | 10 kΩ |
| 7. | DC Supply | 5V - 1A |
| 8. | Mini USB Cable | 1 meter |

# Arduino Nano as ISP Programmer:

The **AT89S52 microcontroller** cannot be programmed directly from a computer or through operating systems such as Windows, Linux, or macOS. To program the AT89S52, **In-System Programming (ISP)** must be used. ISP converts the serial data sent from the computer into **SPI instructions** that the AT89S52 can understand. In this project, an **Arduino Nano** is used as an ISP programmer to perform this conversion. A basic program for configuring the Arduino Nano as an ISP programmer is provided below:

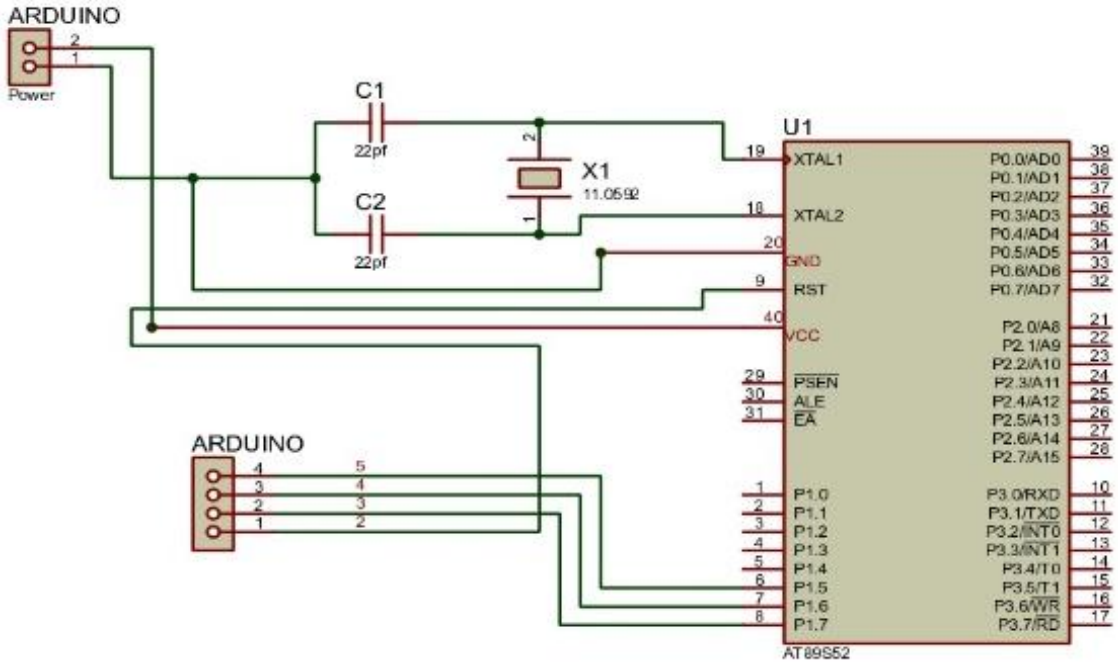
```
#define dummyData 0xAA
#define RDY 75
#define NRDY 76

const int _MISO = 4;
const int _MOSI = 5;
const int _CLK = 3;
const int RST = 2;

/* Variable definition block */

byte data;
byte AL,AH; // 16-bit address
byte lockByte; // embed lock bits here
byte SigH,SigL; // Signature Bytes

void setup()
{
pinMode(_MISO, INPUT);
pinMode(_MOSI, OUTPUT);
pinMode(_CLK, OUTPUT);
pinMode(RST, OUTPUT);
Serial.begin(115200); // depends on the setting of the host PC

}



void loop()
{
  while (!Serial.available()); // wait for character
        if (Serial.available() > 0)
        switch (Serial.read())
        {
```

```
        case 'p': Serial.write(progEnable());
                  break;
        case 'r': readProgmem();
                  Serial.write(data);
                  break;
        case 'a': while(!Serial.available());
                  AL = Serial.read();
                  break;
        case 'A': while(!Serial.available());
                  AH = Serial.read();
                  break;
        case 'd': while(!Serial.available());
                  data = Serial.read();
                  break;
        case 'S': AH = 0;
                  AL = 0;
                  SigH = readSign();
                  Serial.write(SigH);
                  break;
        case 's': AH = 2;
                  AL = 0;
                  SigL = readSign();
                  Serial.write(SigL);
                  AH = 1;
                  AL = 0;
                  SigL = readSign();
                  Serial.write(SigL);
                  break; // read SigL
        case 'o': digitalWrite(RST,1);break;
        case 'c': digitalWrite(RST,0);break;
        case 'e': eraseChip();
                  Serial.write(RDY);
                  break;
        case 'j': break;
        case 'w': writeProgmem();
                  break;
        }



}


unsigned char SendSPI(unsigned char data)
{
  uint8_t retval = 0;
  uint8_t intData = data;
  int t;
```

```
    for (int ctr = 0; ctr < 7; ctr++)
      {
         if (intData & 0x80) digitalWrite(_MOSI,1);
            else digitalWrite(_MOSI,0);

        digitalWrite(_CLK,1);
        delayMicroseconds(1);

        t = digitalRead(_MISO);
        digitalWrite(_CLK,0);

        if (t) retval |= 1; else retval &= 0xFE;
            retval<<=1;
            intData<<= 1;
        delayMicroseconds(1);
    }


  if (intData & 0x80) digitalWrite(_MOSI,1);
    else digitalWrite(_MOSI,0);

    digitalWrite(_CLK,1);
    delayMicroseconds(1);

    t = digitalRead(_MISO);
    digitalWrite(_CLK,0);

    if (t) retval |= 1;
    else retval &= 0xFE;

    return retval;
}


byte progEnable()
{
    SendSPI(0xAC);
    SendSPI(0x53);
    SendSPI(dummyData);

   return SendSPI(dummyData);
}

void eraseChip()
{
     SendSPI(0xAC);
     SendSPI(0x9F);
     SendSPI(dummyData);
     SendSPI(dummyData);
```

```
     delay(520);
}

void readProgmem()
{

    SendSPI(0x20);
    SendSPI(AH);
    SendSPI(AL);
    data = SendSPI(dummyData);
}

void writeProgmem()
{
    SendSPI(0x40);
    SendSPI(AH);
    SendSPI(AL);
    SendSPI(data);
}

void writeLockBits()
{
    SendSPI(0xAC);
    SendSPI(lockByte);
    SendSPI(dummyData);
    SendSPI(dummyData);
}

void readLockBits()
{
    SendSPI(0x24);
    SendSPI(dummyData);
    SendSPI(dummyData);
    lockByte = SendSPI(dummyData);
}

byte readSign()
{
    SendSPI(0x28);
    SendSPI(AH);
    SendSPI(AL);
    return SendSPI(dummyData);
}
```

# Circuit Diagram:




Connections:

| Sr. | Arduino Pin | AT89S52 Pin |
|---|---|---|
| 1. | 2 | 9 (RESET) |
| 2. | 3 | 8 (P1.7) |
| 3. | 4 | 7 (P1.6) |
| 4. | 5 | 6 (P1.5) |
| 5. | 5V | 40 (VCC) |
| 6. | Ground | 20 (GND) |

# Procedure:

1. **Collect the Required Components**
   The components required for the project were gathered. These include:

   - AT89S52 microcontroller
   - Arduino Nano
   - Breadboard and jumper wires

- 11.0592 MHz crystal oscillator
- Two 22 pF capacitors
- 10 kΩ resistor for reset pull-up
- Computer system with USB cable

2. **Install the Programming Software:**
   The software required to program the AT89S52 was copied to the computer (8051 SPI Programmer).

3. **The project folder contains:**
   - **8051 programmer.exe** – the software used to communicate with the Arduino Nano.
   - **AT89S52.ino** – the Arduino sketch that converts serial commands from the computer into SPI instructions for the AT89S52.

   Find files at the end of the document.

4. **Upload the Programmer Code to Arduino Nano:**
   - The **Arduino IDE** was opened on the computer.
   - The file **AT89S52.ino** was opened from the project folder.
   - The Arduino Nano was connected to the computer using a USB cable.
   - The correct **board type and COM port** were selected in the Arduino IDE.
   - The sketch was compiled and uploaded to the Arduino Nano.

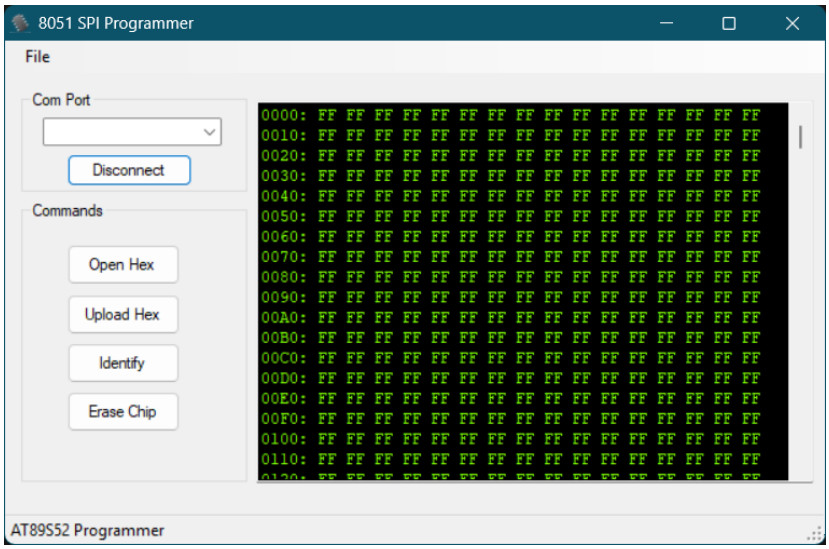
- After uploading, the Arduino Nano acts as an **ISP programmer** for the AT89S52.

5. **Connect the AT89S52 to Arduino Nano:**
   The AT89S52 was connected to the Arduino Nano using SPI communication lines.

6. **Connect the Crystal Oscillator:**
   - An **11.0592 MHz crystal** was connected between pins **18 (XTAL2)** and **19 (XTAL1)** of the AT89S52.
   - Two **22 pF capacitors** were connected from each crystal pin to ground.

7. **Reset Circuit Connection:**
   - A **10 kΩ resistor** was connected between the **RESET pin (Pin 9)** and **VCC** to ensure proper reset operation.

8. **Run the Programming Software:**



- The application **8051 programmer.exe** was opened from the project folder.
- The correct **COM port** corresponding to the Arduino Nano was selected.
- The HEX file of the program to be written to the AT89S52 was loaded.

9. **Program the AT89S52:**
   The software sent serial commands to the Arduino Nano. The Arduino Nano converted these commands into SPI instructions and transmitted them to the AT89S52. The microcontroller was then successfully programmed through In-System Programming (ISP).

10. **Verification of Programming:**
    After programming, the software verified the written data to ensure that the program was correctly stored in the microcontroller's flash memory.

## Skills developed:

1. **Microcontroller Programming Skills** – Developed the ability to write, compile, and upload programs for the 8051 microcontroller using Assembly or Embedded C.
2. **Understanding of In-System Programming (ISP)** – Learned how ISP allows programming of a microcontroller directly on the development board without removing the chip from the circuit.
3. **Embedded System Development Skills** – Gained practical experience in working with embedded systems and

understanding how software interacts with microcontroller hardware.

4. **HEX File Generation and Uploading** – Learned the process of converting source code into a HEX file using a compiler and transferring it to the microcontroller through an ISP programmer.
5. **Hardware Handling and Troubleshooting** – Developed skills in connecting hardware components, identifying programming errors, and solving basic issues during programming and testing.
6. **Computer–Microcontroller Communication** – Understood how a computer communicates with the microcontroller through USB or serial interface during program downloading.

## Applications:

1. **Educational Purposes** – Used in laboratories and colleges to learn microcontroller programming and embedded system concepts.
2. **Embedded System Development** – Helps developers easily program and update code in 8051-based embedded systems.
3. **Rapid Prototyping** – Allows quick testing and modification of programs during the development of electronic projects.
4. **Industrial Control Systems** – Used for programming microcontrollers used in automation, control circuits, and monitoring systems.

5. **Electronics Project Development** – Useful for students and hobbyists to upload programs into microcontroller-based projects such as LED control, motor control, and sensor-based systems.

## Conclusion:

This microproject helped in understanding the working and programming of the 8051 microcontroller using an In-System Programming (ISP) programmer. It demonstrated how a program written in Assembly or Embedded C can be converted into a HEX file and uploaded to the microcontroller through a computer. The project provided practical knowledge about microcontroller programming, hardware connection, and embedded system development. Overall, it made the programming process easier and improved understanding of how microcontrollers are used in electronic and automation applications.

## Files and Dependencies:

Scan the below QR code for all the required files and dependencies.

Thank you!